ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Ejercicios Tema 3 SCD: Paso de mensajes

Índice

1. Ejercicio 77 **3**
1.1. Enunciado 3
1.2. Solucion 3

2. Ejercicio 78 **5**
2.1. Enunciado 5
2.2. Solución 6

3. Ejercicio 79 **7**
3.1. Enunciado 7
3.2. Solución 7

4. Ejercicio 80 **8**
4.1. Enunciado 8
4.2. Solución 9

5. Ejercicio 81 **11**
5.1. Enunciado 11
5.2. Solución 12

6. Ejercicio 82 **12**
6.1. Enunciado 12
6.2. Solución 13

7. Ejercicio 83 **14**
7.1. Enunciado 14
7.2. Solución 15

8. Ejercicio 84 **15**
8.1. Enunciado 15
8.2. Solución 16

9. Ejercicio 85 **17**
9.1. Enunciado 17
9.2. Solución 18

1 Ejercicio 77

1.1. Enunciado

En un sistema distribuido, 6 procesos clientes necesitan sincronizarse de forma específica para realizar cierta tarea, de forma que dicha tarea sólo podrá ser realizada cuando tres procesos estén preparados para realizarla. Para ello, envían peticiones a un proceso controlador del recurso y esperan respuesta para poder realizar la tarea específica. El proceso controlador se encarga de asegurar la sincronización adecuada. Para ello, recibe y cuenta las peticiones que le llegan de los procesos, las dos primeras no son respondidas y producen la suspensión del proceso que envía la petición (debido a que se bloquea esperando respuesta) pero la tercera petición produce el desbloqueo de los tres procesos pendientes de respuesta. A continuación, una vez desbloqueados los tres procesos que han pedido (al recibir respuesta), inicializa la cuenta y procede cíclicamente de la misma forma sobre otras peticiones. El código de los procesos clientes aparece aquí abajo. Los clientes usan envío asíncrono seguro para realizar su petición, y esperan con una recepción síncrona antes de realizar la tarea:

```
process Cliente[ i : 0..5 ] ;
begin
  while true do begin
    send( peticion, Controlador );
    receive( permiso, Controlador );
    Realiza_tarea_grupal();
  end
end

process Controlador ;
begin
  while true do begin
    ...
  end
end
```

Describir en pseudocódigo el comportamiento del proceso controlador, utilizando una orden de espera selectiva que permita implementar la sincronización requerida entre los procesos. Es posible utilizar una sentencia del tipo select for i=... to ... para especificar diferentes ramas de una sentencia selectiva que comparten el mismo código dependiente del valor de un índice i.

1.2. Solucion

En un sistema distribuido, 6 procesos clientes necesitan sincronizarse de forma específica para realizar cierta tarea, de forma que dicha tarea sólo podrá ser realizada cuando tres procesos estén preparados para realizarla. Para ello, envían peticiones a un proceso controlador del recurso y esperan respuesta para poder realizar la tarea específica. El proceso controlador se encarga de asegurar la sincronización adecuada.

El controlador cuenta las peticiones que le llegan de los procesos:

- Las dos primeras peticiones no son respondidas, lo que provoca que los procesos que las envían queden bloqueados esperando una respuesta.

- La tercera petición produce el desbloqueo de los tres procesos pendientes, enviándoles una respuesta.

Una vez desbloqueados los tres procesos, el controlador inicializa la cuenta y procede cíclicamente de la misma forma para nuevas peticiones. Los clientes usan envío asíncrono seguro para realizar su petición y esperan con una recepción síncrona antes de realizar la tarea.

El código de los procesos clientes es el siguiente:

```
process Cliente[ i : 0..5 ] ;
begin
  while true do begin
    send( peticion, Controlador );   // Envía una petición al proceso controlador.
    receive( permiso, Controlador ); // Espera un permiso del controlador.
    Realiza_tarea_grupal();          // Realiza la tarea grupal una vez recibido el permiso.
  end
end
```

Listing 1: Código de los procesos clientes

El comportamiento del proceso controlador, que asegura la sincronización, se describe en pseudocódigo a continuación:

```
process Controlador ;
var
  contador : integer := 0;           // Cuenta las peticiones recibidas.
  buffer : array[0..2] of integer;   // Almacena los índices de los procesos en espera.

begin
  while true do begin
    select
      for i := 0 to 5                // Itera sobre las peticiones de los procesos clientes.
      when receive( peticion, Cliente[i] ) do
        buffer[contador] := i;       // Almacena el índice del proceso cliente en el buffer.
        contador := contador + 1;    // Incrementa el contador de peticiones.

        if contador = 3 then begin   // Si se han recibido tres peticiones ...
          for j := 0 to 2 do
            send( permiso, Cliente[buffer[j]] ); // Envía permiso a los tres procesos.
          contador := 0;             // Reinicia el contador.
        end
    end
  end
end
```

Listing 2: Pseudocódigo del proceso Controlador

Explicación del código del controlador:

- El proceso controlador mantiene un contador de peticiones y un buffer que almacena los índices de los clientes en espera.
- Por cada petición recibida, almacena el índice del cliente en el buffer y aumenta el contador.
- Cuando el contador alcanza tres, el controlador envía permisos a los tres procesos almacenados en el buffer y reinicia el contador.

Este enfoque asegura que los clientes se sincronizan correctamente antes de realizar la tarea grupal, tal como se requiere en el enunciado.

2 Ejercicio 78

2.1. Enunciado

En un sistema distribuido, 3 procesos productores producen continuamente valores enteros y los envían a un proceso buffer que los almacena temporalmente en un array local de 4 celdas enteras para ir enviándoselos a un proceso consumidor. A su vez, el proceso buffer realiza lo siguiente, sirviendo de forma equitativa al resto de procesos:

- Envía enteros al proceso consumidor siempre que su array local tenga al menos dos elementos disponibles.
- Acepta envíos de los productores mientras el array no esté lleno, pero no acepta que cualquier productor pueda escribir dos veces consecutivas en el búfer.

El código del productor y del consumidor es el siguiente:

```
process Productor[ i : 0..2 ] ;
var dato : integer ;
begin
  while true do begin
    dato := Producir();
    send( dato, Buffer );
  end
end

process Consumidor ;
begin
  while true do begin
    receive ( dato, Buffer );
    Consumir( dato );
  end
end

process Buffer ;
begin
  while true do begin
    ...
  end
end
```

Se pide: describir en pseudocódigo el comportamiento del proceso buffer, utilizando una orden de espera selectiva que permita implementar la sincronización requerida entre los procesos.

2.2. Solución

```
Process Buffer;
var
  buffer : array[0..3] of integer; // Array local de 4 celdas enteras.
  count : integer := 0;             // Contador de elementos en el buffer.
  lastProducer : integer := -1;     // Índice del último productor que escribió en el buffer.

  begin

    while true do begin

      select
        when receive(dato, Productor[0]) do
          if(count <4 and lastProducer != 0) then
            buffer[count] := dato; // Almacena el dato en el buffer.
            count := count + 1;   // Incrementa el contador de elementos.
            lastProducer := 0; // Actualiza el índice del último productor.
          end

        when receive(dato, Productor[1]) do
          if(count <4 and lastProducer != 1) then
            buffer[count] := dato; // Almacena el dato en el buffer.
            count := count + 1;   // Incrementa el contador de elementos.
            lastProducer := 1; // Actualiza el índice del último productor.
          end

        when receive(dato, Productor[2]) do
          if(count <4 and lastProducer != 2) then
            buffer[count] := dato; // Almacena el dato en el buffer.
            count := count + 1;   // Incrementa el contador de elementos.
            lastProducer := 2; // Actualiza el índice del último productor.
          end

        when count >= 2 do // Si hay al menos dos elementos en el buffer...
          send(buffer[0], Consumidor); // Envía el primer elemento al consumidor.
          for i := 0 to 2 do
            buffer[i] := buffer[i + 1]; // Desplaza los elementos restantes.
          end
          count := count - 1; // Decrementa el contador de elementos.


```

```
      end


    end
```

Listing 3: Pseudocódigo del proceso Buffer

3 Ejercicio 79

3.1. Enunciado

Suponer un proceso productor y 3 procesos consumidores que comparten un buffer acotado de tamaño B. Cada elemento depositado por el proceso productor debe ser retirado por todos los 3 procesos consumidores para ser eliminado del buffer. Cada consumidor retirará los datos del buffer en el mismo orden en el que son depositados, aunque los diferentes consumidores pueden ir retirando los elementos a ritmo diferente unos de otros.

Por ejemplo, mientras un consumidor ha retirado los elementos 1, 2 y 3, otro consumidor puede haber retirado solamente el elemento 1. De esta forma, el consumidor más rápido podría retirar hasta B elementos más que el consumidor más lento.

Describir en pseudocódigo el comportamiento de un proceso que implemente el buffer de acuerdo con el esquema de interacción descrito usando una construcción de espera selectiva, así como el del proceso productor y de los procesos consumidores. Comenzar identificando qué información es necesario representar, para después resolver las cuestiones de sincronización.

Una posible implementación del buffer mantendría, para cada proceso consumidor, el puntero de salida y el número de elementos que quedan en el buffer por consumir:

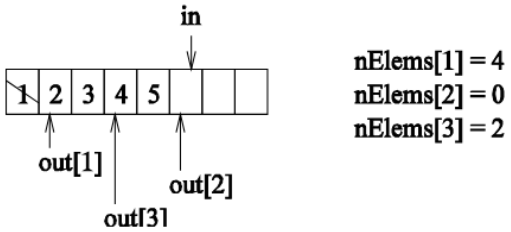

Figura 1: Esquema de interacción entre el productor y los consumidores.

3.2. Solución

```
process Buffer;

var
  out: array[0..2] of integer; // Punteros de salida para cada consumidor
      .
  nElems: array[0..2] of integer; // Número de elementos que quedan por
      consumir para cada consumidor.
```

```

    B: integer; // Tamaño del buffer.
    buf: array[0..B-1]; // Buffer acotado de tamaño B.
    begin
    select for i:=0 to 2 do

      when nElemens[i] != 0 do // Si el consumidor i tiene elementos por consumir...
        send (buf[out[i]], consumidor[i]); // Envía el elemento al consumidor i.
        out[i] + 1 mod B; // Actualiza el puntero de salida del consumidor i.
        nElemens[i] = nElemens[i]-1; // Decrementa el número de elementos por consumir del consumidor i.
      end do

      when nElemens[0] != B or nElemens[1] != B or nElemens[2] != B do // Si el buffer no está lleno para algún consumidor...
        receive(dato, productor); // Recibe un dato del productor.
        bufer[i] = dato; // Almacena el dato en el buffer.
        for j:=0 to 2 do
          nElemens[j] = nElemens[j] + 1; // Incrementa el número de elementos por consumir para cada consumidor.
        end do
      end do
    end select
  end
```

Listing 4: Pseudocódigo del proceso Buffer

Explicación del código del proceso Buffer

El proceso Buffer se encarga de gestionar la interacción entre un productor y tres consumidores que comparten un buffer acotado de tamaño B. Cada elemento depositado por el productor debe ser retirado por todos los consumidores para ser eliminado del buffer. La idea general del código es mantener un buffer acotado de tamaño B donde el productor puede depositar elementos y los consumidores pueden retirarlos. Cada consumidor tiene su propio puntero de salida y contador de elementos por consumir. El proceso Buffer asegura que cada elemento depositado por el productor sea retirado por todos los consumidores antes de ser eliminado del buffer. Además, se asegura de que el buffer no se llene completamente para ningún consumidor y que los consumidores retiren los elementos en el mismo orden en el que fueron depositados.

4 Ejercicio 80

4.1. Enunciado

Implementación de los procesos Salvajes y Cocinero

Una tribu de antropófagos comparte una olla en la que caben M misioneros. Cuando algún salvaje quiere comer, se sirve directamente de la olla, a no ser que ésta esté

vacía. Si la olla está vacía, el salvaje despertará al cocinero y esperará a que éste haya rellenado la olla con otros M misioneros.

```
process Salvaje[ i : 0..2 ] ;
begin
    var peticion : integer := ... ;
    begin
        while true do begin
            // esperar a servirse un misionero
            s_send( peticion, Olla );
            // comer:
            Comer();
        end
    end
end

process Cocinero ;
begin
    while true do begin
        // dormir esperando solicitud para llenar
        // ...
        // confirmar que se ha rellenado la olla
        // ...
    end
end
```

Implementar los procesos salvajes y cocinero usando paso de mensajes, utilizando un proceso Olla que incluye una construcción de espera selectiva que sirve peticiones de los salvajes y el cocinero para mantener la sincronización requerida, teniendo en cuenta que:

- La solución no debe producir interbloqueo.
- Los salvajes podrán comer siempre que haya comida en la olla.
- Solamente se despertará al cocinero cuando la olla esté vacía.

4.2. Solución

```
process Olla;

var
    comida: integer := 0; // Cantidad de comida disponible en la olla.
    capacidad: integer := M; // Capacidad máxima de la olla.
    peticiones: queue of integer; // Cola para gestionar las peticiones de
        los salvajes.

begin
    select
        when not peticiones.empty() and comida > 0 do
            // Atender a un salvaje que quiere comer
            salvaje := peticiones.pop();
            comida := comida - 1;
            send("comer", salvaje);
```

```
        end

        when comida = 0 do
            // Solicitar al cocinero que rellene la olla
            send("rellenar", cocinero);
            receive("rellenado", cocinero);
            comida := capacidad;
        end
    end select
end
```

Listing 5: Pseudocódigo del proceso Olla

```
process Salvaje[i: 0..N-1];

begin
    while true do
        // Solicitar comida a la olla
        send(i, Olla);
        receive("comer", Olla);
        Comer();
    end
end
```

Listing 6: Pseudocódigo del proceso Salvaje

```
process Cocinero;

begin
    while true do
        // Esperar solicitud para rellenar la olla
        receive("rellenar", Olla);
        // Rellenar la olla
        Rellenar();
        send("rellenado", Olla);
    end
end
```

Listing 7: Pseudocódigo del proceso Cocinero

Esta solución asegura:

- Sincronización adecuada entre salvajes y cocinero mediante paso de mensajes.
- Los salvajes pueden comer siempre que haya comida en la olla.
- El cocinero solo se despierta cuando la olla esté vacía.
- Evita el interbloqueo utilizando una cola para gestionar las peticiones de los salvajes.

5 Ejercicio 81

5.1. Enunciado

Considerar un conjunto de N procesos, $P[i]$, $(i = 0, \ldots, N-1)$ que se pasan mensajes cada uno al siguiente (y el primero al último), en forma de anillo. Cada proceso tiene un valor local almacenado en su variable local `mi_valor`. Deseamos calcular la suma de los valores locales almacenados por los procesos de acuerdo con el algoritmo que se expone a continuación.

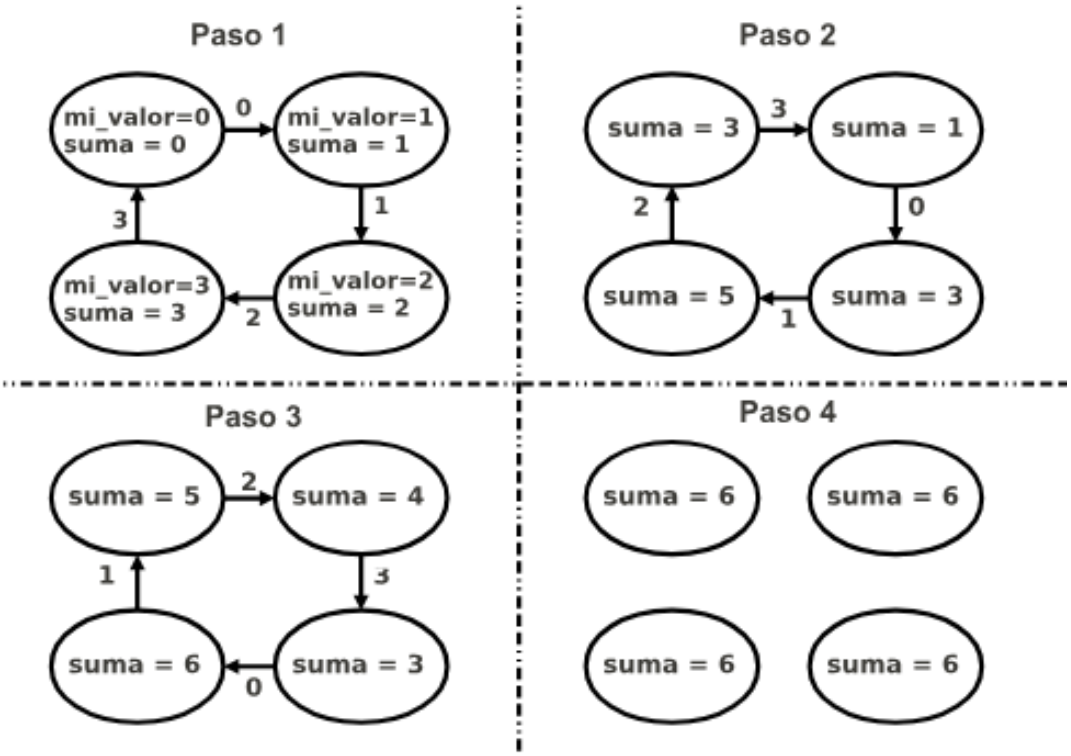

Figura 2: Esquema de interacción entre los procesos.

Los procesos realizan una serie de iteraciones para hacer circular sus valores locales por el anillo. En la primera iteración, cada proceso envía su valor local al siguiente proceso del anillo, al mismo tiempo que recibe del proceso anterior el valor local de éste. A continuación acumula la suma de su valor local y el recibido desde el proceso anterior. En las siguientes iteraciones, cada proceso envía al siguiente proceso siguiente el valor recibido en la anterior iteración, al mismo tiempo que recibe del proceso anterior un nuevo valor. Después acumula la suma. Tras un total de $N-1$ iteraciones, cada proceso conocerá la suma de todos los valores locales de los procesos.

Dar una descripción en pseudocódigo de los procesos siguiendo un estilo SPMD y usando operaciones de envío y recepción síncronas:

```
process P[ i : 0..N-1 ] ;
var mi_valor : integer := ... ; // valor arbitrario (== i en la figura, por
    ejemplo)
    suma : integer := mi_valor ; // suma inicializada a mi_valor
```

```
begin
    for j := 0 to N-1 do begin
        ...
    end
end
```

5.2. Solución

En este caso se afirma que se debe de seguir un estilo SPMD (Single Program Multiple Data), lo que significa que todos los procesos ejecutan el mismo código, pero con datos diferentes. En este caso, cada proceso $P[i]$ tiene un valor local *mi_valor* que se suma a la suma total.

```
  process P[ i : 0..N-1 ] ;
  var mi_valor : integer := i ; // valor arbitrario (== i en la figura, por
      ejemplo)
      suma : integer := mi_valor ; // suma inicializada a mi_valor
  begin
      for j := 0 to N-1 do begin
          send( mi_valor, P[ (i + 1) mod N ] );
          receive( valor_recibido, P[ (i - 1 + N) mod N ] );
          suma := suma + valor_recibido;
          mi_valor := valor_recibido;
      end
  end
```

6 Ejercicio 82

6.1. Enunciado

Considerar un estanco en el que hay tres fumadores y un estanquero. Cada fumador continuamente lía un cigarro y se lo fuma. Para liar un cigarro, el fumador necesita tres ingredientes: tabaco, papel y cerillas. Uno de los fumadores tiene solamente papel, otro tiene solamente tabaco, y el otro tiene solamente cerillas. El estanquero tiene una cantidad infinita de los tres ingredientes.

El estanquero coloca aleatoriamente dos ingredientes diferentes de los tres que se necesitan para hacer un cigarro, desbloquea al fumador que tiene el tercer ingrediente y después se bloquea. El fumador seleccionado se puede obtener fácilmente mediante una función `genera_ingredientes` que devuelve el índice (0, 1, ó 2) del fumador escogido.

El fumador desbloqueado toma los dos ingredientes del mostrador, desbloqueando al estanquero, lía un cigarro y fuma durante un tiempo. El estanquero, una vez desbloqueado, vuelve a poner dos ingredientes aleatorios en el mostrador, y se repite el ciclo.

Describir una solución distribuida que use envío asíncrono seguro y recepción síncrona para este problema usando un proceso Estanquero y tres procesos fumadores Fumador(i) (con $i = 0, 1$ y 2).

```
process Estanquero ;
begin
    while true do begin
        // El estanquero coloca dos ingredientes aleatorios en el mostrador
        // y desbloquea al fumador que tiene el tercer ingrediente
        ...
    end
end

process Fumador[ i : 0..2 ] ;
begin
    while true do begin
        // El fumador espera hasta que sea desbloqueado
        // Toma los dos ingredientes del mostrador, lía un cigarro y fuma
        ...
    end
end
```

6.2. Solución

Usando envío asíncrono seguro y recepción síncrona. Para ello debemos de usar MPI_Isend y MPI_Recv.

```
  process Estanquero ;
  begin
      while true do begin
          // El estanquero coloca dos ingredientes aleatorios en el
              mostrador
          int ingrediente1, ingrediente2;
          ingrediente1 = generarIngrediente();
          do{
              ingrediente2 = generarIngrediente();
          }
          while(ingrediente1 == ingrediente2);
          // y desbloquea al fumador que tiene el tercer ingrediente
          MPI_Isend(ingrediente1, 1, MPI_INT, Fumador[3 - ingrediente1 -
              ingrediente2], 0, MPI_COMM_WORLD);
          MPI_Isend(ingrediente2, 1, MPI_INT, Fumador[3 - ingrediente1 -
              ingrediente2], 0, MPI_COMM_WORLD);
      end
  end

  process Fumador[ i : 0..2 ] ;
  begin
      while true do begin
          // El fumador espera hasta que sea desbloqueado
          int ingrediente1, ingrediente2;
          MPI_Recv(ingrediente1, 1, MPI_INT, Estanquero, 0, MPI_COMM_WORLD,
               MPI_STATUS_IGNORE);
          MPI_Recv(ingrediente2, 1, MPI_INT, Estanquero, 0, MPI_COMM_WORLD,
               MPI_STATUS_IGNORE);
          // Toma los dos ingredientes del mostrador, lía un cigarro y fuma
          LiarCigarro();
          Fumar();
```

```
        end
    end
```

7 Ejercicio 83

7.1. Enunciado

En un sistema distribuido, un gran número de procesos clientes usa frecuentemente un determinado recurso y se desea que puedan usarlo simultáneamente el máximo número de procesos. Para ello, los clientes envían peticiones a un proceso controlador para usar el recurso y esperan respuesta para poder usarlo (véase el código de los procesos clientes). Cuando un cliente termina de usar el recurso, envía una solicitud para dejar de usarlo y espera respuesta del Controlador. El proceso controlador se encarga de asegurar la sincronización adecuada imponiendo una única restricción por razones supersticiosas: nunca habrá 13 procesos exactamente usando el recurso al mismo tiempo.

```
process Cli[ i : 0....n ] ;
var pet_usar : integer := +1 ;
    pet_liberar : integer := -1 ;
    permiso : integer := ... ;
begin
    while true do begin
        send( pet_usar, Controlador );
        receive( permiso, Controlador );
        Usar_recurso( );
        send( pet_liberar, Controlador );
        receive( permiso, Controlador );
    end
end
```

```
process Controlador ;
begin
    while true do begin
        select
            ...
        end
end
```

Describir en pseudocódigo el comportamiento del proceso controlador, utilizando una orden de espera selectiva que permita implementar la sincronización requerida entre los procesos. Es posible utilizar una sentencia del tipo select for i=... to ... para especificar diferentes ramas de una sentencia selectiva que comparten el mismo código dependiente del valor de un índice i.

7.2. Solución

```
process Controlador ;
var contador: integer;
begin
    while true do begin
        select
          for i:= 0 to n do

            when receive(pet_usar,Cli[i]) do
              if contador < 13 then
                send(permiso,Cli[i]);
                contador := contador + 1;
              end if

            when receive(pet_liberar,Cli[i]) do
              send(permiso,Cli[i]);
              contador := contador - 1;
            end do

        end select
    end
end
```

Lógica de la solución

En este caso se ha pensado la solución de manera simple. Se trata de un controlador que recibe peticiones de los clientes para usar un recurso. Si el contador de procesos que están usando el recurso es menor que 13, el controlador envía un permiso al cliente para que pueda usar el recurso. Cuando un cliente termina de usar el recurso, envía una petición para liberarlo y el controlador disminuye el contador. De esta forma, se asegura que nunca haya exactamente 13 procesos usando el recurso al mismo tiempo. Es impotante **uso del if** para asegurarnos de que solo se usan los 13 procesos a la vez, y debemos de asegurarnos de que actualizamos de manera correcta la variable contador.

8 Ejercicio 84

8.1. Enunciado

En un sistema distribuido, tres procesos Productor se comunican con un proceso Impresor que se encarga de ir imprimiendo en pantalla una cadena con los datos generados por los procesos productores. Cada proceso productor (`Productor[i]` con $i = 0, 1, 2$) genera continuamente el correspondiente entero i, y lo envía al proceso Impresor.

El proceso Impresor se encarga de ir recibiendo los datos generados por los productores y los imprime por pantalla (usando el procedimiento `imprime(entero)`) generando una cadena de dígitos en la salida. No obstante, los procesos se han de sincronizar adecuadamente para que la impresión por pantalla cumpla las siguientes restricciones:

- Los dígitos 0 y 1 deben aceptarse por el impresor de forma alterna. Es decir, si se acepta un 0 no podrá volver a aceptarse un 0 hasta que se haya aceptado un 1, y viceversa, si se acepta un 1 no podrá volver a aceptarse un 1 hasta que se haya aceptado un 0.
- El número total de dígitos 0 o 1 aceptados en un instante no puede superar el doble de número de dígitos 2 ya aceptados en dicho instante.

Cuando un productor envía un dígito que no se puede aceptar por el impresor, el productor quedará bloqueado esperando completar el s_send. El pseudocódigo de los procesos productores (Productor) se muestra a continuación, asumiendo que se usan operaciones bloqueantes no buferizadas (síncronas):

```
process Productor[ i : 0,1,2 ]
while true do begin
    s_send( i, Impresor ) ;
end
```

```
process Impresor
  var
      .....
    begin
      while true do begin
        select
          .....
        end
      end
    end
```

Listing 8: Pseudocódigo del proceso Impresor

8.2. Solución

```
  process Impresor
  var
      contador_0_1 : integer := 0 ;
      contador_2 : integer := 0 ;
      turno : integer := 0 ; // 0: turno para aceptar 0, 1: turno para
         aceptar 1
  begin
      while true do begin
          select
            for i:= 0 to 2
              when receive(dato, Productor[i]) do
                  if (dato == 0 and turno == 0 and contador_0_1 < 2 *
                     contador_2) then
                      imprime(dato);
                      contador_0_1 := contador_0_1 + 1;
                      turno := 1;
                  else if (dato == 1 and turno == 1 and contador_0_1 < 2 *
                     contador_2) then
                      imprime(dato);
                      contador_0_1 := contador_0_1 + 1;
```

```
                        turno := 0;
                    else if (dato == 2) then
                        imprime(dato);
                        contador_2 := contador_2 + 1;
                    end if;
                end when
            end select
        end
    end
```

Lógica de la solución

En este código del proceso `Impresor`, se maneja la sincronización entre los productores y el impresor usando un `select`, y además diseñandolo la solución usando la opción for para hacer más simple el código. Para cada productor, se controla la aceptación de los dígitos 0, 1 y 2, con las restricciones mencionadas en el enunciado. El proceso acepta un dígito según las condiciones del turno (alternancia entre 0 y 1) y el límite sobre el número de dígitos 0 y 1 que pueden aceptarse en comparación con los dígitos 2.

9 Ejercicio 85

9.1. Enunciado

En un sistema distribuido hay un vector de n procesos iguales que envían con `send` (en un bucle infinito) valores enteros a un proceso receptor, que los imprime. Si en algún momento no hay ningún mensaje pendiente de recibir en el receptor, este proceso debe imprimir "no hay mensajes, duermo"; después de bloquearse durante 10 segundos (con `sleep_for(10)`), antes de volver a comprobar si hay mensajes (esto podría hacerse para ahorrar energía, ya que el procesamiento de mensajes se hace en ráfagas separadas por 10 segundos).

Este problema no se puede solucionar usando `receive` o `i_receive`. Indica a qué se debe esto. Sin embargo, sí se puede hacer con `select`. Diseña una solución a este problema con `select`:

```
process Emisor[ i : 1..n ]
var dato : integer ;
begin
    while true do begin
        dato := Producir() ;
        send( dato, Receptor );
    end
end
```

```
process Receptor()
var dato : integer ;
begin
    while true do
        ......
end
```

9.2. Solución